# Architecting Automated Execution on Polymarket: A Comprehensive CLOB V2 Python Reference

The evolution of decentralized finance has birthed highly specialized microstructures, none more prominent in the predictive domain than Polymarket. Operating at the intersection of blockchain consensus and high-frequency trading, Polymarket facilitates binary prediction markets through a unique hybrid-decentralized architecture. This architecture splits the trading lifecycle into two distinct phases: off-chain order matching via a Central Limit Order Book (CLOB), and on-chain trade settlement via smart contracts deployed on the Polygon network[1].
For quantitative developers and algorithmic trading architects, building robust execution modules requires mastering both traditional financial market structures and cryptographic Web3 authentication patterns. This exhaustive technical report provides a rigorous, beginner-friendly blueprint for constructing an automated execution engine targeting Polymarket's CLOB V2 API. The analysis specifically explores the lifecycle of 5-Minute BTC Up/Down binary options, focusing entirely on Maker Limit Orders implemented via the official py-clob-client-v2 Python SDK[3].
By traversing cryptographic authentication (EIP-712 and ERC-1271), token allowance mechanics, floating-point precision constraints, and dynamic risk management through High-Water Mark (HWM) tracking, this report equips the modern algo-developer with a production-grade execution framework.

## The Hybrid-Decentralized Microstructure

Before inspecting the Python implementation, it is vital to understand the venue's underlying plumbing. Polymarket does not utilize an Automated Market Maker (AMM) like Uniswap; instead, it relies on a Central Limit Order Book (CLOB) for price discovery and matching[1].
An intuitive analogy for this hybrid architecture is a restaurant tab. When patrons order drinks throughout the evening, the bartender notes the orders on a ledger behind the bar. This process represents the off-chain limit orders. It is instantaneous, incurs no immediate credit card processing fees (gas fees), and allows patrons to cancel or change their minds before the drink is poured. However, at the end of the night, the final tally is processed through the credit card machine, cementing the transactions permanently with the bank. This represents the on-chain settlement.
In the Polymarket ecosystem, traders sign digital authorization slips (off-chain orders) using their private keys. The Polymarket matching engine pairs opposing buyers and sellers who possess complimentary views on an event[2]. Once a match is found, the matching engine submits the paired cryptographic signatures to the CTFExchange smart contract on the Polygon blockchain, which locks collateral and mints the corresponding "YES" and "NO" Conditional Token Framework (CTF) outcome tokens[1]. This guarantees non-custodial security

while circumventing the high latency and gas fees associated with placing every resting order directly on the blockchain.

# SECTION 1: Client Initialization & Metamask Authentication

To interface with the CLOB V2, the trading module must navigate a complex, two-tiered authentication system. The legacy architecture of V1, represented by the py-clob-client package, has been entirely deprecated and will reject incoming API requests[4]. Any modern integration must rely strictly on the py-clob-client-v2 package[3].

## The Two-Level Authentication Paradigm

Authentication in Polymarket is strictly divided into Level 1 (L1) and Level 2 (L2)[3]. This dual-layer approach separates the fundamental ownership of the funds from the high-frequency trading credentials, minimizing the risk of private key exposure during rapid execution.

1. **Level 1 (L1) Auth - Wallet Signature:** This is the cryptographic baseline. The application uses the wallet's actual private key to sign an EIP-712 standard typed data message[3]. Returning to our everyday analogy, L1 authentication is akin to presenting a government-issued passport at a high-security hotel front desk. It proves absolute identity but is too cumbersome (and risky) to present at every single door inside the building. The L1 signature is used strictly to request or derive a set of high-frequency API credentials[7].
2. **Level 2 (L2) Auth - HMAC API Credentials:** Once the L1 signature is validated, the Polymarket server issues or derives an API Key, API Secret, and API Passphrase (L2 credentials)[3]. These act as the VIP hotel keycard. The L2 credentials are used to sign HTTP requests via a Hash-based Message Authentication Code (HMAC) for high-frequency actions like placing orders, fetching balances, and cancelling resting limits[3].

## Metamask Web3 Wallets and Signature Types

When utilizing a standard Metamask Web3 Wallet (Externally Owned Account or EOA), the developer must configure the correct signature_type. Polymarket supports multiple cryptographic verification pathways depending on how the user's funds are custodied[4].

| Signature Type Variable | Integer Value | Description and Use Case |
|---|---|---|
| EOA | 0 | Standard Ethereum wallet (e.g., Metamask, Hardware Wallet). Funds reside directly on the EOA. |
| POLY_PROXY | 1 | Legacy proxy wallet for Magic Link / Email logins. |

| | | |
|---|---|---|
| POLY_GNOSIS_SAFE | 2 | Gnosis Safe multisig proxy wallet. |
| POLY_1271 | 3 | Modern Deposit Wallet (ERC-1271). Frequently deployed for new UI-connected Metamask users. |

For a standard Metamask EOA that holds its own USDC.e or pUSD directly, the system utilizes signature_type=0. However, the Polymarket platform has increasingly migrated toward counterfactual proxy wallets—meaning a user connects their Metamask, but Polymarket deploys a specialized smart contract (Deposit Wallet) to hold the funds and handle gasless approvals[10].

If the user's Polymarket profile indicates a distinct "Proxy Address" or "Deposit Wallet" that differs from their Metamask address, the algorithm must specify signature_type=3 (POLY_1271) and explicitly declare the funder address (the address of the proxy contract holding the funds) during ClobClient initialization[10]. The L1 authentication is still signed by the Metamask private key, but the API recognizes that the funds and order execution will route through the specified funder smart contract[13].

## Mandatory Token Allowances and Smart Contracts

In Web3 mechanics, decentralized applications and exchange contracts cannot arbitrarily pull funds from a wallet. The wallet owner must explicitly grant permission through an ERC-20 or ERC-1155 approve transaction on the blockchain[5]. This is the digital equivalent of writing a pre-authorized blank check to a specific merchant.

Before an automated algorithm can place orders, the Web3 wallet (or its proxy) must hold a sufficient balance of the base collateral token—typically USDC.e or wrapped pUSD—and must have granted infinite allowances to the Polymarket exchange smart contracts[4].

| Contract Name | Polygon Contract Address | Asset Type | Purpose |
|---|---|---|---|
| **USDC.e** | 0x2791Bca1f2de4661ED88A30C99A7a9449Aa84174 | ERC-20 | The primary stablecoin collateral required for trading[5]. |
| **CTF Exchange** | 0x4bFb41d5B3570 | Smart Contract | The primary |

| | | | |
|---|---|---|---|
| **(V1/V2)** | DeFd03C39a9A4D8dE6Bd8B8982E | | matching engine settlement contract for binary standard markets[5]. |
| **Neg Risk Adapter** | 0xd91E80cF2E7be2e162c6513ceD06f1dD0dA35296 | Smart Contract | The adapter contract handling mutually exclusive outcome resolutions[5]. |
| **Conditional Tokens** | 0x4D97DCd97eC945f40cF65F87097ACe5EA0476045 | ERC-1155 | The actual outcome token framework representing the YES/NO shares[5]. |

For a Metamask user, these allowances are typically generated by interacting with the Polymarket UI at least once to trigger the blockchain approval pop-ups[10]. However, the off-chain CLOB matching engine maintains its own internal cache of these blockchain balances to minimize latency. If the algorithm deposits funds but fails to command the API to synchronize this cache, the CLOB will reject perfectly valid orders with a 400: not enough balance / allowance error[6].

To prevent this desynchronization, the Python module must invoke the update_balance_allowance endpoint after initializing the L2 credentials[20].

## Python Implementation: Client Setup Snippet

The following code demonstrates the meticulous initialization process, mapping the abstract architectural concepts into a concrete Python implementation using py-clob-client-v2. Note that while earlier documentation referenced create_or_derive_api_creds, the updated V2 client utilizes create_or_derive_api_key()[8].

```
Python
import os
from py_clob_client_v2 import ClobClient, ApiCreds
from dotenv import load_dotenv

# Load environment variables (Private Key for Metamask)
load_dotenv()
METAMASK_PK = os.getenv("PRIVATE_KEY")
HOST = "https://clob.polymarket.com"
```

```
CHAIN_ID = 137 # Polygon Mainnet ID

def initialize_polymarket_client(is_proxy: bool = False, funder_address: str = None) -> ClobClient:
    """
    Initializes the ClobClient, demonstrating the Level 1 to Level 2 authentication pipeline.
    Handles both standard EOA (Metamask) and Deposit Wallet (Proxy) configurations.
    """
    # Determine the correct signature type
    # 0 = EOA (Standard Web3 Wallet), 3 = POLY_1271 (Deposit Wallet Proxy)
    sig_type = 3 if is_proxy else 0

    print("[*] Initiating Level 1 (L1) Wallet Authentication...")

    # Step 1: Instantiate a temporary client using the Metamask private key for L1 Auth
    temp_client = ClobClient(
        host=HOST,
        chain_id=CHAIN_ID,
        key=METAMASK_PK,
        signature_type=sig_type,
        funder=funder_address if is_proxy else None
    )

    # Step 2: Derive Level 2 (L2) API Credentials
    # This automatically signs the EIP-712 payload and requests HMAC credentials from the CLOB
    derived_creds = temp_client.create_or_derive_api_key()

    # Map the returned dictionary-like structure to the ApiCreds class
    creds = ApiCreds(
        api_key=derived_creds.api_key,
        api_secret=derived_creds.api_secret,
        api_passphrase=derived_creds.api_passphrase
    )

    # Step 3: Initialize the final, fully-authenticated trading client
    print("[*] Level 1 Auth successful. Constructing Level 2 Trading Client...")
    trading_client = ClobClient(
        host=HOST,
        chain_id=CHAIN_ID,
        key=METAMASK_PK,
        creds=creds,
        signature_type=sig_type,
        funder=funder_address if is_proxy else None
    )
```

```
    # Step 4: Synchronize the CLOB Cache with the On-Chain Blockchain Reality
    # This ensures the off-chain matching engine recognizes our USDC.e/pUSD balances and
allowances
    print("[*] Synchronizing CLOB balance and allowance cache...")
    trading_client.update_balance_allowance(asset_type="COLLATERAL")

    print("[+] Client initialization complete and ready for execution.")
    return trading_client
```

# SECTION 2: Order Structure & Maker Execution Mechanics

With a fully authenticated and state-synchronized client, the trading module must navigate the mechanics of the limit order book. The 5-minute BTC Up/Down markets are characterized by high volatility, fleeting liquidity, and rapid order book updates.

## Market Microstructure: Maker vs. Taker Dynamics

In modern order book microstructures, participants are fundamentally divided into Makers and Takers.

- **The Maker:** Operates like an artisan placing goods in a shop window with a clearly marked price tag. They provide inventory (liquidity) to the market by placing Limit Orders that sit on the order book. They wait passively for a counterparty.
- **The Taker:** Operates like an aggressive customer walking into the store and buying whatever is currently on the shelf. They consume liquidity by crossing the bid-ask spread to execute immediately.

To mitigate slippage and control execution costs strictly, this execution module is constrained to Maker limit orders. By utilizing OrderType.GTC (Good-Til-Cancelled), the algorithm posts a resting limit order that will remain on the book until it is matched by a taker or explicitly cancelled by the client[9].

## The OrderArgs Payload and Order Lifecycle

To construct a Maker limit order in the V2 Python client, the application packages the trade parameters into the OrderArgs data class[7].

| Parameter | Type | Description |
|---|---|---|
| token_id | String | The unique 77-digit integer identifying the specific outcome token (e.g., the YES share for BTC Up)[8]. |

| | | |
|---|---|---|
| price | Float | The fractional probability cost, bounded strictly between $0.00 and $1.00 [5]. |
| size | Float | The total number of shares to purchase or sell[8]. |
| side | Enum | Imported from Side.BUY or Side.SELL, denoting the trade direction[8]. |

Rather than manually constructing the payload, hashing it locally, signing it, and then submitting it via two separate functions (create_order and post_order), algorithmic developers should utilize the create_and_post_order macro[7]. This streamlined function handles the EIP-712 typed data construction, the local cryptographic signing using the private key, and the subsequent REST API transmission in a single, atomic Python call, reducing internal code latency.

## The IEEE-754 Floating-Point Drift Hazard

An insidious and highly documented edge case in programmatic trading involves floating-point precision, particularly when interacting with blockchain smart contracts that lack native floating-point types[25].

Polymarket prediction tokens trade in discrete tick sizes of $0.01$. However, standard Python floats are stored in base-2 binary, meaning a conceptually simple number like $16.90$ is actually stored in memory as $16.89999999999999858$ [25].

When the py-clob-client-v2 internally calculates the required collateral (Size $\times$ Price), this micro-drift can result in an unrounded fraction. The CLOB matching engine enforces a strict two-decimal limit on collateral amounts. If the internal calculation results in a tertiary decimal place (e.g., maker % 10000 != 0), the matching engine will instantly reject the order, throwing a validation error[25].

To prevent this, the algorithmic architect must meticulously sanitize and quantize all size and price variables using Python's Decimal module before passing them into the OrderArgs payload. This guarantees absolute string-to-decimal fidelity. Furthermore, when placing the order, the developer must explicitly define the tick size constraint using PartialCreateOrderOptions(tick_size="0.01")[7].

## Entry Strategy Implementation: Momentum Breakout

The algorithm's entry strategy targets 5-Minute BTC Up/Down tokens based on a short-term momentum surge. Because the market resolves in just 5 minutes, these tokens exhibit

hyper-sensitivity to the underlying Binance BTC/USDT oracle spot price[26].

**The Condition:**

The strategy monitors the live order book. A valid entry signal occurs if the target token's midpoint price (the average of the best bid and best ask) increases by $\geq \$0.15$ within a rolling 10-second window, provided the current absolute price remains $> \$0.10$.

**The Execution Math:**

To strictly guarantee Maker status while capturing the asset before the momentum fades, the algorithm must post an order slightly behind the current market frontier.

If the current Best Ask is $P_{ask}$, the target Maker Entry Price ($P_{entry}$) is calculated as:

$$P_{entry} = P_{ask} - \$0.02$$

By posting a Limit Buy at $\$0.02$ below the lowest willing seller, the order does not immediately cross the spread. It rests safely on the bid side of the book. This avoids Taker fees, guarantees exact price execution, and waits for a localized mean-reversion or a motivated seller to hit the bid.

## Python Implementation: Maker Entry Snippet

```python
from decimal import Decimal, ROUND_DOWN
from py_clob_client_v2 import OrderArgs, OrderType, PartialCreateOrderOptions, Side

def quantize_decimal(value: float, decimal_places: int = 2) -> float:
    """
    Mitigates IEEE-754 floating-point binary drift (e.g., 16.89999998)
    by forcing absolute precision truncation required by the CLOB engine.
    """
    quantizer = Decimal('1').scaleb(-decimal_places)
    return float(Decimal(str(value)).quantize(quantizer, rounding=ROUND_DOWN))

def calculate_maker_entry(best_ask_price: float) -> float:
    """
    Calculates the Maker entry price exactly $0.02 behind the Best Ask.
    """
    # Ensure the price doesn't drop below the absolute floor of $0.01
    return max(quantize_decimal(best_ask_price - 0.02), 0.01)
```

```
def execute_maker_buy(client, token_id: str, entry_price: float, size: float) -> str:
    """
    Constructs the OrderArgs and dispatches the Limit Buy to the CLOB.
    """
    print(f"[*] Constructing Maker BUY order for {size} shares at ${entry_price}")

    # Construct the strictly typed OrderArgs payload
    order_payload = OrderArgs(
        token_id=token_id,
        price=entry_price,
        size=size,
        side=Side.BUY
    )

    # Dispatch via the macro method, enforcing tick size limits
    response = client.create_and_post_order(
        order_args=order_payload,
        options=PartialCreateOrderOptions(tick_size="0.01"),
        order_type=OrderType.GTC
    )

    order_id = response.get("orderID")
    print(f"[+] Maker BUY successfully posted. Order ID: {order_id}")
    return order_id
```

# SECTION 3: Exit Management & Native Order Cancellations

A professional algorithmic architecture dedicates substantially more mathematical rigor and computational resources to risk management and position exits than it does to trade entry. For the Polymarket execution module, risk is managed asymmetrically via a static Take Profit (TP) and a dynamic, state-aware Trailing Stop Loss (SL).

## Take Profit (TP) Logic

The objective of the Take Profit mechanism is to secure a defined yield while avoiding the hard upper boundary of the outcome token architecture. Polymarket tokens are binary; they ultimately resolve at exactly $\$1.00$ (if the oracle condition is met) or $\$0.00$ (if it fails)[2]. Upon a successful entry fill, the algorithm immediately computes the target price:

$$P_{tp} = \min(\text{Entry Price} + \$0.20, \$0.995)$$

The mathematical min() function ensures that even if the entry occurs late in the momentum cycle at $\$0.85$, the algorithm will not attempt to set a Take Profit at $\$1.05$. Attempting to post a price $> \$1.00$ is structurally impossible and would trigger an immediate API validation error[5].

By intentionally capping the exit at $\$0.995$, the algorithm locks in the profit just before the final resolution, bypassing the opportunity cost of holding locked collateral until official oracle settlement, which can sometimes be delayed. The algorithm executes this by placing a resting OrderType.GTC Limit Sell order at $P_{tp}$.

## High-Water Mark (HWM) and Trailing Stop Loss (SL)

While the Take Profit rests passively on the order book, the Stop Loss requires active, dynamic management. Because the 5-Minute BTC market can reverse violently, a static stop loss is insufficient. Instead, the algorithm employs a High-Water Mark (HWM) tracking system.
The HWM is an internal state variable maintained within the Python polling loop. It records the absolute highest market bid price the token has achieved since the algorithm entered the trade. It acts as a mechanical ratchet—moving up as the price rises in our favor, but never coming back down.

The dynamic Stop Loss trigger price ($P_{sl}$) is continually recalculated inside the loop as:

$$P_{sl} = \max(\text{Entry Price} - \$0.10, \text{HWM} - \$0.10)$$

If the live market bid price drops to or below $P_{sl}$, the algorithm must enact an emergency exit protocol to cut losses and protect capital.

## The Cancellation and Egress Protocol

Executing the Stop Loss introduces a complex state management problem. Before the algorithm can dump its inventory to trigger the Stop Loss, it must reclaim its locked token balance. When the resting Take Profit Limit Sell order was placed, the CLOB matching engine locked the algorithm's tokens so they could not be double-spent.
To execute the Stop Loss, the system must perform a precise, atomic sequence:

1. **Cancel the resting Take Profit order** natively using client.cancel(order_id) or client.cancel_order()[20]. This command routes to the CLOB and instantly releases the locked shares back into the proxy wallet.
2. **Verify cancellation** by awaiting the HTTP 200 success response to avoid insufficient balance errors on the next step.
3. **Execute the Stop Loss** by placing a new Limit Sell order at the current Best Bid to liquidate the position immediately. (While the prompt strictly requires Maker orders, in a severe stop-loss scenario, crossing the spread is functionally required to guarantee exit, mimicking a Market order via a deeply priced Limit Sell).

It is crucial to consider the REST API rate limits during this loop. The POST /order and DELETE /order endpoints allow up to 5,000 requests per 10 seconds (burst) and 120,000 per 10 minutes (sustained)[11]. While generous, an unthrottled while True polling loop can easily exhaust these limits, leading to IP-level blocking. The polling loop must feature a disciplined sleep interval (e.g., 1 second).

## Python Implementation: Exit Management Snippet

```
Python
import time

def manage_position_lifecycle(client, token_id: str, entry_price: float, size: float, tp_order_id: str):
    """
    Manages the active position by tracking the High-Water Mark (HWM).
    Monitors for Take Profit fills, and executes a trailing Stop Loss if required.
    """
    # Initialize the High-Water Mark at our entry price
    high_water_mark = entry_price
    position_active = True

    print(f"[*] Initiating Position Monitoring. Entry: ${entry_price} | TP Order ID: {tp_order_id}")

    while position_active:
        time.sleep(1)  # Strict 1-second throttle to respect API rate limits

        # Step 1: Check if the resting Take Profit order was hit
        tp_order_info = client.get_order(tp_order_id)
        if tp_order_info and tp_order_info.get("status") == "MATCHED":
            print("[+] Take Profit Hit! Position Closed Successfully.")
            position_active = False
            break

        # Step 2: Fetch the live order book to update the HWM
        current_book = client.get_order_book(token_id)
        # Safely extract the best bid, defaulting to 0.00 if the bid side is empty
        current_bid = float(current_book.bids[0].price) if current_book.bids else 0.00

        # Step 3: Update HWM if current bid exceeds the previous record
        if current_bid > high_water_mark:
            high_water_mark = current_bid
```

```
            print(f"[*] Price moving in favor. New High-Water Mark: ${high_water_mark}")

        # Step 4: Calculate the dynamic Trailing Stop Loss trigger
        # Ratchets up with the HWM, but never drops below the initial floor (Entry - $0.10)
        sl_trigger = quantize_decimal(max(entry_price - 0.10, high_water_mark - 0.10))

        # Step 5: Evaluate Stop Loss Condition
        if current_bid <= sl_trigger and current_bid > 0.00:
            print(f"[-] Trailing Stop Loss Triggered at ${current_bid}. Commencing Egress Sequence.")

            # Egress Phase A: Cancel resting Take Profit to free locked inventory
            print("[*] Cancelling resting TP order to release locked shares...")
            cancel_resp = client.cancel(tp_order_id)

            # Egress Phase B: Post Limit Sell at current bid to exit immediately
            sl_args = OrderArgs(
                token_id=token_id,
                price=quantize_decimal(current_bid),
                size=size,
                side=Side.SELL
            )

            print(f"[*] Dispatching Stop Loss Exit at ${quantize_decimal(current_bid)}...")
            client.create_and_post_order(
                order_args=sl_args,
                options=PartialCreateOrderOptions(tick_size="0.01"),
                order_type=OrderType.GTC
            )
            print("[+] Stop Loss execution complete. Position liquidated.")
            position_active = False
```

# SECTION 4: Complete Step-by-Step Execution Code Flow

To synthesize the complex architectural concepts, cryptography, floating-point math, and game-theoretic risk management discussed above, this section provides an end-to-end, runnable reference script.

This production-grade script strings together the initialization protocols, the momentum entry signal verification, the Limit Buy Maker placement, and the unified Take Profit / Trailing Stop Loss active monitoring loop.

*Prerequisites:* Ensure the environment has py-clob-client-v2 and python-dotenv installed, and that the .env file contains the private key[4]. For this demonstration, we assume a standard

Metamask EOA holding its own funds (signature_type=0), but proxy logic is available.

```python
"""
Polymarket CLOB V2 Automated Execution Module
Target: 5-Minute BTC Up/Down Binary Options
Strategy: Momentum Breakout Entry with HWM Trailing Stop Loss
"""

import os
import time
from decimal import Decimal, ROUND_DOWN
from py_clob_client_v2 import ClobClient, ApiCreds, OrderArgs, OrderType, PartialCreateOrderOptions, Side
from dotenv import load_dotenv

# ==========================================
# ENVIRONMENT & CONSTANT CONFIGURATION
# ==========================================
load_dotenv()
PK = os.getenv("PRIVATE_KEY")
HOST = "https://clob.polymarket.com"
CHAIN_ID = 137  # Polygon Mainnet
# Example Token ID for a hypothetical BTC 5-Min Up outcome token
DYNAMIC_ASSET_ID = "58438108339183552152169312788007524148185983754158488881158329175661908109855"

# ==========================================
# UTILITY FUNCTIONS
# ==========================================
def quantize_decimal(value: float, decimals: int = 2) -> float:
    """
    Mitigates IEEE-754 floating-point drift (e.g., 16.89999)
    by quantizing to strictly two decimal places for CLOB validation compliance.
    """
    q = Decimal('1').scaleb(-decimals)
    return float(Decimal(str(value)).quantize(q, rounding=ROUND_DOWN))

# ==========================================
# PHASE 1: CLIENT INITIALIZATION & AUTH
# ==========================================
def initialize_client() -> ClobClient:
```

```
    """
    Executes the 2-Tier Authentication Protocol.
    Initializes EIP-712 L1 Auth to derive L2 HMAC credentials.
    """
    print("[*] Initializing L1 Authentication (Metamask EOA)...")

    # Temporary client for L1 Auth (signature_type=0 for standard EOA)
    temp_client = ClobClient(
        host=HOST,
        chain_id=CHAIN_ID,
        key=PK,
        signature_type=0
    )

    # Derive L2 Credentials
    creds_dict = temp_client.create_or_derive_api_key()
    creds = ApiCreds(
        api_key=creds_dict.api_key,
        api_secret=creds_dict.api_secret,
        api_passphrase=creds_dict.api_passphrase
    )

    # Initialize the final trading client
    print("[*] Initializing L2 Trading Client...")
    client = ClobClient(
        host=HOST,
        chain_id=CHAIN_ID,
        key=PK,
        creds=creds,
        signature_type=0
    )

    # Sync CLOB Cache with On-Chain pUSD/CTF Allowances
    print("[*] Syncing Balance and Allowances with CLOB cache...")
    client.update_balance_allowance(asset_type="COLLATERAL")

    return client

# ==========================================
# PHASE 2 & 3: MAIN EXECUTION ENGINE
# ==========================================
def execute_trading_module(client: ClobClient, target_token_id: str, order_size: float):
    """
    Main execution loop handling Entry, HWM Tracking, TP, and Trailing SL.
```

```
    """
    print(f"[*] Commencing observation for Token ID: {target_token_id}")

    entry_filled = False
    entry_price = 0.0

    # Track momentum (simplified rolling price for demonstration)
    last_mid_price = quantize_decimal(client.get_midpoint(target_token_id))

    # ----------------------------------------
    # ENTRY SIGNAL TRACKING LOOP
    # ----------------------------------------
    while not entry_filled:
        time.sleep(1)  # Respecting 500/s API rate limits

        current_book = client.get_order_book(target_token_id)
        best_ask = float(current_book.asks[0].price) if current_book.asks else 1.00
        current_mid = quantize_decimal(client.get_midpoint(target_token_id))

        momentum_jump = current_mid - last_mid_price

        # Condition: Jump >= $0.15 AND Base Price > $0.10
        if momentum_jump >= 0.15 and current_mid > 0.10:
            print(f"[!] Momentum Signal Detected. Jump: +${quantize_decimal(momentum_jump)}.
Calculating Maker Entry.")

            # Maker Execution: Post limit at Best Ask - $0.02
            target_entry_price = max(quantize_decimal(best_ask - 0.02), 0.01)

            # Construct Maker Limit Buy Order
            order_args = OrderArgs(
                token_id=target_token_id,
                price=target_entry_price,
                size=quantize_decimal(order_size),
                side=Side.BUY
            )

            print(f"[*] Posting Limit BUY at ${target_entry_price}")
            resp = client.create_and_post_order(
                order_args=order_args,
                options=PartialCreateOrderOptions(tick_size="0.01"),
                order_type=OrderType.GTC
            )
```

```
            # Simulate waiting for the Maker order to fill
            entry_order_id = resp.get("orderID")
            while True:
                status = client.get_order(entry_order_id).get("status")
                if status == "MATCHED":
                    entry_price = target_entry_price
                    entry_filled = True
                    print(f"[+] Entry Maker Order Filled at ${entry_price}")
                    break
                time.sleep(1)

        last_mid_price = current_mid

    # ----------------------------------------
    # EXIT MANAGEMENT (TP & TRAILING SL)
    # ----------------------------------------

    # Calculate Take Profit: min(Entry + $0.20, $0.995)
    tp_price = quantize_decimal(min(entry_price + 0.20, 0.995))

    print(f"[*] Posting Resting Take Profit (SELL) at ${tp_price}")
    tp_args = OrderArgs(
        token_id=target_token_id,
        price=tp_price,
        size=quantize_decimal(order_size),
        side=Side.SELL,
    )

    tp_resp = client.create_and_post_order(
        order_args=tp_args,
        options=PartialCreateOrderOptions(tick_size="0.01"),
        order_type=OrderType.GTC,
    )
    tp_order_id = tp_resp.get("orderID")

    # Initialize High-Water Mark (HWM) tracker
    high_water_mark = entry_price
    position_active = True

    while position_active:
        time.sleep(1)  # Rate limit throttle
```

```
        # Check if the resting TP order was hit and filled
        tp_status = client.get_order(tp_order_id).get("status")
        if tp_status == "MATCHED":
            print("[+] Take Profit Hit! Position Closed.")
            position_active = False
            break

        current_book = client.get_order_book(target_token_id)
        current_bid = float(current_book.bids[0].price) if current_book.bids else 0.00

        # Update HWM if current bid exceeds previous record
        if current_bid > high_water_mark:
            high_water_mark = current_bid
            print(f"[*] New High-Water Mark Reached: ${high_water_mark}")

        # Calculate dynamic Trailing Stop Loss trigger
        sl_trigger = quantize_decimal(max(entry_price - 0.10, high_water_mark - 0.10))

        # Evaluate Stop Loss condition
        if current_bid <= sl_trigger and current_bid > 0.00:
            print(f"[-] Trailing Stop Loss Triggered at ${current_bid}. Commencing Egress.")

            # Step 1: Cancel resting Take Profit to free locked shares
            print("[*] Cancelling resting TP order...")
            client.cancel(tp_order_id)

            # Step 2: Post Limit Sell at current bid to exit immediately
            sl_args = OrderArgs(
                token_id=target_token_id,
                price=quantize_decimal(current_bid),
                size=quantize_decimal(order_size),
                side=Side.SELL
            )

            client.create_and_post_order(
                order_args=sl_args,
                options=PartialCreateOrderOptions(tick_size="0.01"
                order_type=OrderType.GTC
            )
            print("[+] Stop Loss execution complete. Position liquidated.")
            position_active = False

# ========================================
```

```
# SCRIPT INVOCATION
# ==========================================
if __name__ == "__main__":
    try:
        poly_client = initialize_client()
        # Execute module with a 10-share trade size
        execute_trading_module(client=poly_client, target_token_id=DYNAMIC_ASSET_ID,
order_size=10.0)
    except Exception as e:
        print(f"[!] Critical Execution Error: {e}")
```

# Architectural Synthesis

By treating the prediction market as a high-frequency, decentralized limit order book rather than a rudimentary betting interface, the quantitative developer unlocks robust programmatic capabilities[26].

The transition to Polymarket's V2 matching engine fundamentally altered the authentication pathways, requiring complex multi-tiered EIP-712 credential derivation and strict handling of the L1 versus L2 authentication models[11]. By systematically handling the initialization cache (update_balance_allowance), aggressively countering Python's inherent binary drift via decimal quantization, and managing risk through concurrent resting and polling loops, this execution module offers a resilient foundation.

Whether responding to microsecond macroeconomic catalysts or managing large-scale inventory across algorithmic portfolios, adhering strictly to Maker execution paradigms ensures absolute cost control. When combined with dynamic exit management loops tracking high-water marks, the resulting software architecture preserves the mathematical integrity of the underlying trading strategies, ensuring alpha is realized securely on-chain.

### Works cited


1. The Anatomy of a Blockchain Prediction Market: Polymarket in the 2024 U.S. Presidential Election - arXiv, https://arxiv.org/html/2603.03136v2
2. The Anatomy of Polymarket: Evidence from the 2024 Presidential Election - arXiv, https://arxiv.org/html/2603.03136v1
3. PY Polymarket CLOB Client V2 - PyPI, https://pypi.org/project/py-clob-client-v2/
4. How to Set Up a Polymarket Bot on a VPS - TradoxVPS, https://tradoxvps.com/how-to-set-up-a-polymarket-bot-on-a-vps/
5. Python client for the Polymarket CLOB - GitHub, https://github.com/Polymarket/py-clob-client
6. Orders rejected with 'invalid order version' — migration guide from py-clob-client v1? #92, https://github.com/Polymarket/py-clob-client-v2/issues/92
7. py-clob-client-v2 1.1.0 on PyPI - Libraries.io, https://libraries.io/pypi/py-clob-client-v2
8. Polymarket/py-clob-client-v2 - GitHub,

https://github.com/Polymarket/py-clob-client-v2
9. web3-polymarket | Skills Marketplace - LobeHub, https://lobehub.com/skills/polyblocks-polyblocks-skills
10. POLY_1271 + Deposit Wallet: "signer address has to be the address of the API KEY" despite matching addresses · Issue #64 · Polymarket/py-clob-client-v2 - GitHub, https://github.com/Polymarket/py-clob-client-v2/issues/64
11. Polymarket V2 migration guide: updating your trading bots for 2026 - TradoxVPS, https://tradoxvps.com/polymarket-v2-migration-guide/
12. Order placement returns 400 "maker address not allowed, please use the deposit wallet flow" for Magic.link proxy wallets (py-clob-client-v2 1.0.0 + 1.0.1) · Issue #56 - GitHub, https://github.com/Polymarket/py-clob-client-v2/issues/56
13. POLY_1271 (sig type 3) order placement fails: L1 auth always binds API key to EOA, never the deposit wallet · Issue #56 · Polymarket/rs-clob-client-v2 - GitHub, https://github.com/Polymarket/rs-clob-client-v2/issues/56
14. POLY_1271 (sig type 3) order placement fails: L1 auth always binds API key to EOA, never the deposit wallet · Issue #70 · Polymarket/py-clob-client-v2 - GitHub, https://github.com/Polymarket/py-clob-client-v2/issues/70
15. py-clob-client - PyPI, https://pypi.org/project/py-clob-client/0.23.0/
16. The Anatomy of a Decentralized Prediction Market: Microstructure Evidence from the Polymarket Order Book - arXiv, https://arxiv.org/html/2604.24366v2
17. Polymarket/ctf-exchange - GitHub, https://github.com/polymarket/ctf-exchange
18. #58438108339183552152169312788007524148185983754158488881158329175661908109855 | Polymarket | PolygonScan, https://polygonscan.com/nft/0x4d97dcd97ec945f40cf65f87097ace5ea0476045/58438108339183552152169312788007524148185983754158488881158329175661908109855
19. CLOB orders failing with "not enough balance / allowance" despite full approvals and correct balance · Issue #287 · Polymarket/py-clob-client - GitHub, https://github.com/Polymarket/py-clob-client/issues/287
20. quantpylib.wrappers.polymarket, https://quantpylib.hangukquant.com/wrappers/polymarket/
21. polymarket开发文档-Central Limit Order Book_polymarket 自动下单 - CSDN博客, https://blog.csdn.net/qq_61786525/article/details/156613775
22. POLY_1271 deposit-wallet orders rejected: 'the order signer address has to be the address of the API KEY' · Issue #75 · Polymarket/py-clob-client-v2 - GitHub, https://github.com/Polymarket/py-clob-client-v2/issues/75
23. Polymarket API: The Complete Developer | Dwellir Guides, https://www.dwellir.com/guides/polymarket-api-guide
24. py-clob-client - PyPI, https://pypi.org/project/py-clob-client/0.1.8/
25. [BUG] Limit orders: round_down in get_order_amounts produces sub-cent amount, server rejects #68 - GitHub, https://github.com/Polymarket/py-clob-client-v2/issues/68
26. OpenMarket: A Synchronized Polymarket–Binance Dataset for High-Frequency Prediction-Market Research - arXiv, https://arxiv.org/html/2607.26245v1
27. polymarket_api文档_概览_polymarket 开源地址 - CSDN博客,

https://blog.csdn.net/qq_61786525/article/details/161089670
28. createApiKey() / create_or_derive_api_key() doesn't EIP-1271-wrap L1 auth for POLY_1271 deposit wallets — orders rejected with signer != api_key · Issue #65 · Polymarket/clob-client-v2 - GitHub, https://github.com/Polymarket/clob-client-v2/issues/65